

C Preprocessor, Macros, and Headers

Based on ECM2433 preprocessor and program-structure material: includes, macros, guards, extern, static, and separate compilation.

Description: What You Need To Know

1. The preprocessor rewrites source before compilation

It handles directives such as `#include`, `#define`, `#ifdef`, and `#ifndef` before the compiler proper sees the translation unit.

```
#include "stack.h"
#define MAGIC 42
```

2. Macro expansion is textual substitution

```
#define PI 3.1415926
#define ELEPHANT "Size of an elephant!"

printf(ELEPHANT); /* macro expands */
printf("ELEPHANT"); /* quoted text does not */
```

3. Function-like macros need care

Macro arguments are substituted as text. Parenthesise parameters and the whole result, and avoid side effects in macro arguments.

```
#define BAD_MAX(a,b) a > b ? a : b
#define MAX(a,b) ((a) > (b) ? (a) : (b))
```

4. Stringification and conditional compilation

```
#define REPORT_FLOAT(X) fprintf(stderr, "%s is %f\n", #X, X)

#ifdef DEBUG
    REPORT_FLOAT(value);
#endif
```

5. Headers, include guards, static, and extern

Headers normally contain declarations and type definitions. Source files contain definitions. Include guards prevent duplicate processing of the same header contents.

```
#ifndef STACK_H
#define STACK_H

typedef struct node *STACK;
extern STACK push(STACK stack, char value);
extern char pop(STACK *stack);

#endif
```

- `static` on a file-scope helper function limits visibility to that source file.
- `extern` declares an object or function defined elsewhere.
- Include guards should avoid reserved identifiers such as names beginning with double underscores.

Practice Questions

- 1 Explain what the C preprocessor does before compilation.
- 2 State what `#include "stack.h"` does in practical terms.
- 3 Write one object-like macro and say how it expands.
- 4 Explain why `#define MAX(a,b) a > b ? a : b` is dangerous with `MAX(3, 2) + 5`.
- 5 Write a safer fully parenthesised version of `MAX(a, b)`.
- 6 Explain why `MAX(i++, j++)` can still be a problem.
- 7 Compare a macro with a function for the same calculation.
- 8 Explain stringification and give a short macro example.
- 9 Write the basic pattern of an include guard for `stack.h`.
- 10 Explain the difference between `static` and `extern` in separate source files.

Mark Scheme

Use this after attempting the questions.

- 1 It rewrites the source before compilation, handling include files, macro substitution, and conditional compilation.
- 2 It copies the contents of the named header into the source file at that point before compilation.
- 3 Example: `#define LIMIT 100`. The name is replaced textually in code.
- 4 Direct substitution changes grouping. The expansion behaves like `3 > 2 ? 3 : 2 + 5`, not like a grouped maximum plus five.
- 5 Expected: `#define MAX(a,b) ((a) > (b) ? (a) : (b))`.
- 6 The chosen argument may be evaluated more than once, so increments or other side effects can happen more than once.
- 7 A function is type-checked and evaluates each argument once. A macro is text substitution and is easier to misuse.
- 8 Stringification uses `#` in a macro body to turn the argument text into a string, such as `#define REPORT(X) printf("%s", #X)`.
- 9 Expected: `#ifndef STACK_H, #define STACK_H`, header contents, then `#endif`.
- 10 `static` limits a file-scope definition to the current source file. `extern` declares something defined elsewhere.